

Fachliches Konzept: Graphbasierte Service-Modellierung für Service-Komposition und Provisionierung

1. Zielsetzung

Dieses Konzept beschreibt, wie Services und deren Bestandteile nicht nur hierarchisch, sondern zusätzlich graphbasiert modelliert werden können, um komplexe Abhängigkeiten, Reihenfolgen und Wiederverwendung fachlich korrekt abzubilden.

Die graphbasierte Service-Modellierung dient dazu,

- komplexe Service-Kompositionen verständlich zu beschreiben
- Abhängigkeiten explizit und auswertbar zu machen
- Provisionierungsreihenfolgen automatisiert abzuleiten
- Parallelisierungspotenziale zu erkennen
- Wiederverwendbare Servicebausteine aufzubauen
- Fehler, Blockaden und Kompensationen fachlich besser steuerbar zu machen

Das Konzept ist fachlich formuliert und unabhängig von konkreten Produkten oder Technologien.

2. Ausgangslage

In vielen Organisationen werden Services zunächst hierarchisch beschrieben. Ein Hauptservice wird in Teilservices zerlegt, diese wiederum in weitere Teilservices. Dieses Modell ist hilfreich, um einen Service strukturell zu verstehen.

Für die eigentliche Provisionierung genügt eine reine Hierarchie jedoch oft nicht.

Typische Probleme sind:

- Ein Teilservice hängt nicht nur von seinem direkten Parent ab, sondern von mehreren anderen Teilservices.
- Ein Basisbaustein wird in mehreren Services wiederverwendet.
- Einzelne Schritte können parallel ausgeführt werden, andere nicht.
- Manche Abhängigkeiten gelten nur unter bestimmten Bedingungen oder Varianten.
- Ein fachlich untergeordneter Service kann technisch eine Voraussetzung für einen fachlich höher eingeordneten Bereich sein.
- Es gibt alternative Bereitstellungspfade.
- Es müssen Blockaden, Wartezustände und Ausnahmefälle explizit modelliert werden.

Daher wird zusätzlich zur hierarchischen Sicht eine graphbasierte Sicht benötigt.

3. Grundidee der graphbasierten Modellierung

Die graphbasierte Modellierung betrachtet einen Service nicht nur als Baum, sondern als **Menge von Knoten und Beziehungen**.

Knoten

Knoten repräsentieren fachliche Einheiten, zum Beispiel:

- Services
- Sub-Services
- Bereitstellungsbausteine
- Vorbedingungen
- Validierungsschritte
- manuelle Aktivitäten
- Kompensationsschritte

Kanten

Kanten repräsentieren Beziehungen zwischen diesen Knoten, zum Beispiel:

- besteht aus
- hängt ab von
- aktiviert
- blockiert
- validiert
- kompensiert
- ist Alternative zu

Dadurch entsteht ein fachliches Netzwerkmodell, das sowohl Struktur als auch Ablaufvoraussetzungen sichtbar macht.

4. Hierarchische und graphbasierte Sicht im Zusammenspiel

4.1 Hierarchische Sicht

Die hierarchische Sicht beantwortet vor allem die Frage:

Woraus besteht ein Service?

Beispiel:

- Service „Benutzerkonto bereitstellen“
 - Sub-Service „Konto anlegen“
 - Sub-Service „Basisattribute setzen“
 - Sub-Service „Mailbox bereitstellen“

Diese Sicht ist gut geeignet für:

- Katalogdarstellung
- Verantwortlichkeiten
- grobe Service-Struktur
- Kommunikation mit Fachbereichen

4.2 Graphbasierte Sicht

Die graphbasierte Sicht beantwortet vor allem die Frage:

Welche Abhängigkeiten und Beziehungen bestehen zwischen den Bestandteilen?

Beispiel:

- „Mailbox bereitstellen“ hängt von „Konto anlegen“ ab
- „Konto aktivieren“ hängt von „Basisattribute setzen“ ab
- „Abschlussvalidierung“ hängt von „Konto bereitstellen“ und „Mailbox bereitstellen“ ab
- „Sonderberechtigung setzen“ ist nur aktiv, wenn Variante X gewählt wurde

Diese Sicht ist gut geeignet für:

- automatische Reihenfolgeermittlung
- Fehleranalyse
- Parallelisierung
- Ausnahmebehandlung
- Wiederverwendung

4.3 Fachliche Schlussfolgerung

Ein Service sollte deshalb **gleichzeitig** in zwei Dimensionen beschreibbar sein:

- **Hierarchie** für Struktur und Verständlichkeit
- **Graph** für Abhängigkeiten und Ausführung

5. Fachliche Modellierungsprinzipien

5.1 Trennung von Struktur und Abhängigkeit

Ein Bestandteil kann fachlich Teil eines Service sein, ohne dass seine Ausführungsreihenfolge allein aus dieser Zugehörigkeit folgt.

Beispiel:

Ein Teilservice gehört organisatorisch zu einem Hauptservice, darf aber erst ausgeführt werden, wenn ein anderer, fachfremd wirkender Basisservice erfolgreich abgeschlossen wurde.

Deshalb müssen folgende Fragen getrennt modelliert werden:

- Zu welchem Service gehört ein Baustein?
- Wovon hängt seine Ausführung ab?

5.2 Wiederverwendbarkeit von Knoten

Ein Sub-Service soll in mehreren Service-Kompositionen verwendet werden können, ohne mehrfach neu modelliert werden zu müssen.

Beispiel:

Ein Basisbaustein „Identität prüfen“ kann in verschiedenen Services vorkommen.

5.3 Explizite Beziehungen statt impliziter Annahmen

Abhängigkeiten dürfen nicht nur im Fliesstext oder in Prozessbeschreibungen versteckt sein. Sie müssen als eigene modellierbare Beziehungen vorliegen.

5.4 Bedingte Aktivierung

Nicht jeder Knoten ist in jedem Fall aktiv. Die Aktivierung kann abhängen von:

- Variante
- Parametern
- Benutzergruppe
- Standort
- Genehmigung
- bestehendem Zielzustand

5.5 Versionierbarkeit

Graphen müssen versioniert werden können, damit nachvollziehbar bleibt, nach welchem Modell ein Service provisioniert wurde.

6. Fachliche Elemente des Servicegraphen

6.1 Knotentypen

Service-Knoten

Repräsentiert einen bestellbaren oder fachlich sichtbaren Service.

Sub-Service-Knoten

Repräsentiert einen Bestandteil eines Service.

Aktivitäts-Knoten

Repräsentiert einen konkreten Bereitstellungsschritt.

Vorbedingungs-Knoten

Repräsentiert eine Voraussetzung, die erfüllt sein muss.

Entscheidungs-Knoten

Repräsentiert eine Regel oder Auswahl, die den weiteren Pfad beeinflusst.

Validierungs-Knoten

Prüft, ob ein gewünschter Zustand erreicht wurde.

Kompensations-Knoten

Beschreibt eine Gegenmassnahme bei Fehlern oder Rueckabwicklung.

Manueller Knoten

Repräsentiert einen menschlichen Bearbeitungsschritt.

6.2 Beziehungstypen

besteht aus

Hierarchische Zerlegung eines Service in Bestandteile.

depends on

Ein Knoten darf erst aktiv werden, wenn ein anderer Knoten erfolgreich abgeschlossen ist.

enables

Ein Knoten schaltet einen anderen frei.

blocks

Ein Knoten oder Zustand verhindert die Ausführung eines anderen.

validates

Ein Knoten prüft das Ergebnis eines anderen.

compensates

Ein Knoten beschreibt die fachliche Rückbehandlung eines anderen.

alternative to

Zwei oder mehr Knoten stehen in einer Alternativbeziehung.

requires condition

Ein Knoten ist nur aktiv, wenn eine bestimmte Bedingung gilt.

7. Beispielhafte Modellierung

7.1 Hierarchische Sicht

Service: Benutzerkonto mit Mailbox bereitstellen

- Konto bereitstellen
- Organisationskontext setzen
- Mailbox bereitstellen
- Abschluss validieren

7.2 Graphsicht

- Konto bereitstellen Voraussetzung für Organisationskontext setzen
- Konto bereitstellen Voraussetzung für Mailbox bereitstellen
- Organisationskontext setzen zusätzliche Voraussetzung für Mailbox bereitstellen
- Mailbox bereitstellen Voraussetzung für Abschluss validieren
- Konto bereitstellen Voraussetzung für Abschluss validieren

Hier wird sichtbar:

Die Struktur allein genügt nicht, um die Reihenfolge korrekt abzuleiten.

8. Warum ein Baum allein nicht genügt

Ein Baum setzt voraus, dass jeder Knoten genau einen übergeordneten Kontext hat und die relevanten Beziehungen entlang dieses Baumes verlaufen.

In echten Service-Landschaften ist das oft nicht der Fall.

Typische Gegenbeispiele:

Mehrfachabhängigkeit

Ein Schritt braucht zwei oder mehr vorgängige Ergebnisse.

Querverbindungen

Ein Teilservice hängt von einem anderen Zweig der Struktur ab.

Wiederverwendung

Ein identischer Baustein wird in mehreren Services eingesetzt.

Bedingungen

Ein Knoten ist nur in bestimmten Fällen aktiv.

Parallelität

Mehrere Knoten können gleichzeitig ausgeführt werden.

Kompensation

Die Rückbehandlung folgt nicht zwingend derselben Struktur wie die Vorwärtsausführung.

Daher ist der Baum für Übersicht hilfreich, aber fachlich nicht ausreichend.

9. Nutzung des Graphen für die Provisionierung

Der Servicegraph ist nicht nur Dokumentation, sondern kann zur Steuerung genutzt werden.

9.1 Ableitung ausführbarer Knoten

Ein Knoten ist ausführbar, wenn:

- er aktiv ist
- alle Pflichtvorbedingungen erfüllt sind
- alle abhängigen Knoten erfolgreich sind
- keine Blockade vorliegt

9.2 Erkennung von Parallelität

Wenn mehrere Knoten keine offenen gegenseitigen Abhängigkeiten haben, können sie parallel ausgeführt werden.

9.3 Erkennung ungültiger Modelle

Das Modell ist fachlich fehlerhaft, wenn:

- Zyklen in harten Abhängigkeiten bestehen
- Pflichtknoten unerreichbar sind
- Varianten zu Widersprüchen führen
- Kompensationslogik unklar ist

9.4 Teilstatus und Blockaden

Der Graph hilft festzustellen, welche Teile eines Service bereits erfolgreich sind und welche noch blockiert oder offen sind.

10. Fachliche Regeln für graphbasierte Modellierung

Für die Modellierung sollten verbindliche Regeln eingeführt werden.

10.1 Eindeutige Identifikation

Jeder Knoten und jede Beziehung braucht eine eindeutige Identifikation.

10.2 Benannte Beziehungstypen

Beziehungen sollen standardisierte Typen verwenden, nicht frei formulierbare Texte.

10.3 Trennung zwischen harten und weichen Abhängigkeiten

- **harte Abhängigkeit:** ohne diese keine Ausführung
- **weiche Abhängigkeit:** fachlich sinnvoll, aber nicht immer blockierend

10.4 Keine versteckten Regeln

Regeln dürfen nicht nur in Prozessdokumenten stehen, sondern müssen am Graphen referenzierbar sein.

10.5 Zyklusfreiheit bei harten Ausführungsabhängigkeiten

Ein Graph aus harten Provisionierungsabhängigkeiten muss azyklisch sein, damit eine Reihenfolge berechnet werden kann.

10.6 Variantenlogik separat ausdrücken

Varianten sollten nicht durch Kopieren des gesamten Graphen modelliert werden, sondern über Aktivierungsregeln oder gezielte Unterschiede.

11. Fachliche Vorteile

Eine graphbasierte Modellierung bietet folgende Vorteile:

Höhere Transparenz

Abhängigkeiten werden sichtbar und nicht nur implizit angenommen.

Automatisierbare Reihenfolgeermittlung

Die Provisionierungslogik kann aus dem Modell abgeleitet werden.

Bessere Wiederverwendung

Sub-Services und Aktivitätsbausteine können mehrfach genutzt werden.

Bessere Fehleranalyse

Blockierte oder fehlerhafte Pfade werden klarer sichtbar.

Skalierbarkeit

Das Modell wächst besser mit der Komplexität als reine Prozesslisten.

Grundlage für Governance

Fachliche Regeln, Verantwortungen und Varianten können kontrollierter gepflegt werden.

12. Herausforderungen

Die graphbasierte Modellierung bringt auch Anforderungen mit sich:

Höhere Modellierungskomplexität

Das Modell ist mächtiger, aber auch anspruchsvoller.

Bedarf an Modellierungsdisziplin

Ohne klare Regeln entstehen unübersichtliche Graphen.

Schulungsbedarf

Fach- und IT-Rollen müssen verstehen, wie Hierarchie und Abhängigkeit zusammenspielen.

Bedarf an guten Visualisierungen

Ein grosser Graph muss verständlich dargestellt werden können, z. B. gefiltert nach Sichtweise.

13. Empfohlene Sichten für die Dokumentation

Für Confluence oder ähnliche Dokumentationen würde ich mehrere Sichten pflegen:

13.1 Service-Struktursicht

Hierarchische Zerlegung des Service.

13.2 Abhängigkeitsgraph

Graphische oder tabellarische Sicht der Knoten und Kanten.

13.3 Variantsicht

Welche Teile gelten für welche Varianten.

13.4 Ausführungssicht

Welche Knoten im Provisionierungskontext relevant sind.

13.5 Ausnahme- und Kompensationssicht

Welche Fehlerpfade und Gegenmassnahmen existieren.

14. Empfohlene Minimalstruktur für ein Modell

Für jeden modellierten Service sollte mindestens dokumentiert werden:

Knotenliste

- Knoten-ID
- Name
- Typ
- Beschreibung
- Pflicht/optional
- Variante
- verantwortliche Rolle

Beziehungsliste

- Von-Knoten

- Nach-Knoten
- Beziehungstyp
- hart/weich
- Bedingung
- Beschreibung

Regeln

- Aktivierungsregeln
 - Validierungsregeln
 - Kompensationsregeln
-

15. Zusammenfassung

Die graphbasierte Service-Modellierung ergänzt die hierarchische Sicht um eine explizite Beschreibung von Abhängigkeiten, Bedingungen und Ausführungslogik.

Ein reines Baum-Modell hilft, Services verständlich zu strukturieren, reicht aber für komplexe Provisionierungszusammenhänge in der Regel nicht aus.

Ein Graph-Modell ermöglicht es, Service-Kompositionen fachlich sauber, wiederverwendbar und automatisierbar zu beschreiben. Es schafft damit eine tragfähige Grundlage für:

- Service-Komposition
- Provisionierungsortchestrierung
- Statussteuerung
- Fehlerbehandlung
- Wiederverwendung
- spätere technische Automatisierung